

Smart Greenhouse Farming IoT project

Documentation

1 Team members

Bodrul Amin Murad

Ismail Hossain

Raihan Uddin

2 Introduction

In the face of increasing global food demand and climate variability, agricultural practices are evolving toward greater efficiency and sustainability. Smart greenhouses represent a modern solution to optimize plant growth through automated environmental monitoring and control. This project focuses on developing a low-cost, IoT-based smart greenhouse prototype using MQTT protocol, microcontrollers, and cloud platforms. The system enables real-time data monitoring and remote interaction with actuators based on sensor feedback, improving water usage and plant care. This report documents the technical implementation and outcomes of the project, demonstrating how embedded systems and IoT technologies can transform traditional farming into a smart, responsive environment.

The development of the smart greenhouse system was carried out through a structured process involving hardware selection, sensor integration, communication setup, software development, and system testing. The following steps outline the methodology employed:

1. System Design and Requirements Definition

The project began by defining the functional requirements of the smart greenhouse, including automated monitoring of temperature, humidity, and soil moisture, and remote control of actuators such as water pumps and ventilation. A system architecture was designed to ensure modularity, scalability, and real-time responsiveness.

2. Hardware Selection and Setup

An **Arduino Uno Rev 2** microcontroller was selected as the core processing unit due to its built-in Wi-Fi capability and ease of integration with sensors and actuators. Key sensors included:

- **DHT11** for temperature and humidity monitoring,
- **Soil moisture sensor** for detecting soil water content,
- **Relay modules** to control water pumps or fans.

These components were assembled on a breadboard and connected to the Arduino for prototyping.

3. **Sensor Calibration and Data Acquisition**

Sensors were calibrated and tested individually to ensure accurate readings. Sensor data was collected at regular intervals and transmitted to the microcontroller for processing.

4. **Communication Protocol Implementation (MQTT)**

The **MQTT protocol** was implemented to facilitate lightweight and efficient communication between the greenhouse system and cloud-based services. The Arduino was programmed to publish sensor data to a public or private MQTT broker and subscribe to control commands. **MQTT Pane** was used as a client interface to monitor topics and send actuator control messages manually during testing.

5. **Cloud and Interface Integration**

MQTT messages were routed through a broker (such as Mosquitto), and the data was visualized using **MQTT Pane**. This allowed real-time observation of environmental conditions and manual control over actuators from any internet-enabled device.

6. **Automation Logic and Feedback Control**

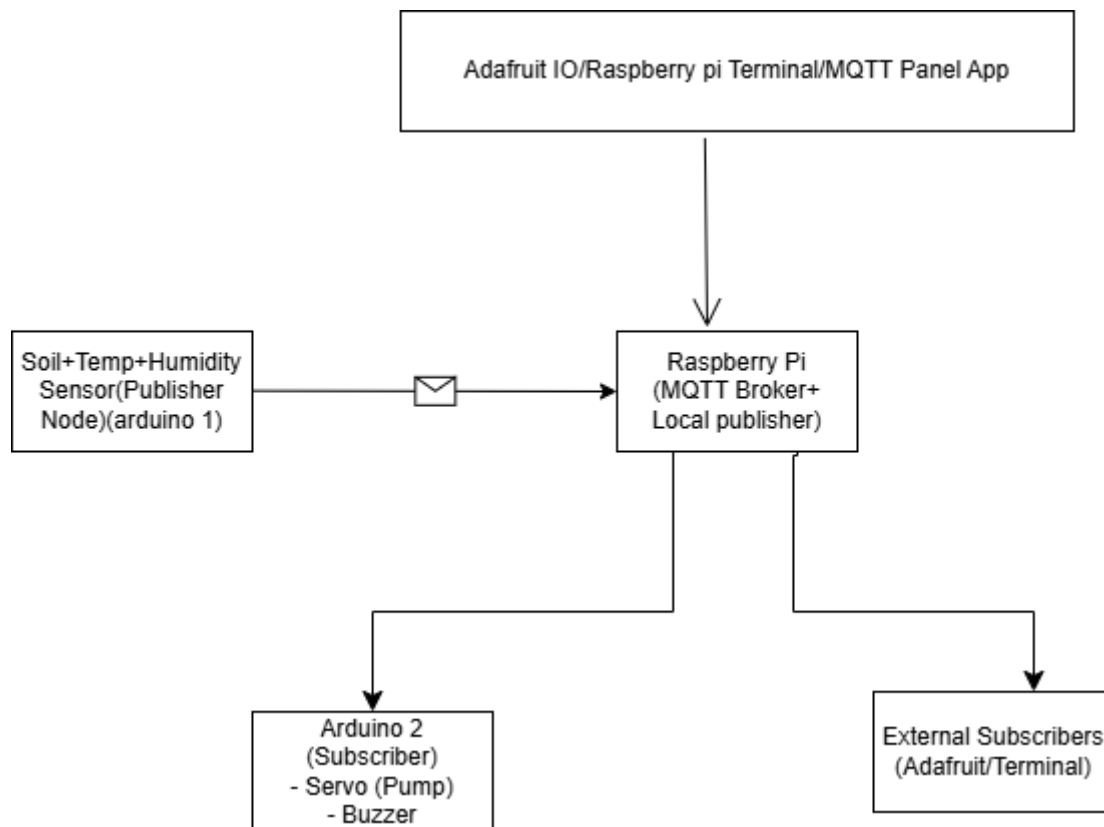
The system was programmed with threshold-based automation logic. For example, when the soil moisture value dropped below a defined threshold, the system would automatically trigger the water pump via the relay module. This closed-loop feedback mechanism ensured dynamic and autonomous operation based on sensor inputs.

7. **Testing and Validation**

The complete system was tested under simulated greenhouse conditions to evaluate its responsiveness, data accuracy, and control reliability. Parameters such as latency, sensor noise, and actuator timing were observed and adjusted to optimize performance.

8. **Documentation and Analysis**

Throughout the project, all stages were documented, and data logs were maintained to analyze the system's effectiveness. The results demonstrate that the integration of MQTT, embedded systems, and IoT principles can significantly enhance the sustainability and efficiency of small-scale greenhouse farming.



3 Project/Team management

Group members	Task 1	Task 2	Task 3	Task 4	Task 5
<i>Bodrul Amin</i> <i>Murad</i>	Project idea	Structure and implementation	Arduino and sensors	Raspberry pi and MQTT	IOT MQTT Panel App
<i>Ismail Hossain</i>	Project idea	Structure and implementation	Arduino and sensors	Raspberry pi and MQTT	IOT MQTT Panel App
Raihan Uddin	Project idea	Structure and implementation	Arduino and sensors	Raspberry pi and MQTT	IOT MQTT Panel App

4 Technologies

The **Arduino Uno Rev 2** (or similar microcontrollers with network capabilities) can act as an MQTT client. Here's how it typically works:

1. Network Connection:

Arduino connects to the internet via built-in Wi-Fi (Rev 2) or an external module like ESP8266/ESP32.

Team Name

2. MQTT Client Library:

Libraries such as PubSubClient are used to implement MQTT functionality on Arduino.

3. Publishing Sensor Data:

Arduino reads from sensors (e.g., DHT11 for temperature/humidity, soil moisture sensors) and publishes the data to MQTT topics.

4. Subscribing to Commands:

Arduino also subscribes to control topics (e.g., `greenhouse/pump`) to receive instructions—like turning a water pump on or off based on remote input or automation.

5. Broker Communication:

The Arduino communicates with a broker (e.g., running on a Raspberry Pi or cloud-based service) to send/receive messages.

The **Raspberry Pi** serves as a powerful and flexible MQTT component. It can act as:

A. MQTT Broker

- Software like **Mosquitto** runs on Raspberry Pi to handle all MQTT traffic.
- It receives messages from publishing clients (e.g., Arduino) and forwards them to all subscribers.
- This allows centralized management of data and device coordination within the greenhouse.

B. MQTT Client

- Raspberry Pi can also act as a client, using Python (with libraries like **paho-mqtt**) to:
 - Subscribe to topics (e.g., to log sensor data from Arduino).
 - Publish control messages (e.g., send a command to Arduino to start irrigation).
 - Interface with a **cloud platform**, database (e.g., InfluxDB), or visualization tool (e.g., Grafana or MQTT Pane).

DHT11: The DHT11 is a low-cost digital sensor used for measuring temperature and humidity. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and it outputs a digital signal on a single data pin. The sensor is suitable for environmental monitoring applications like smart greenhouses due to its simplicity and reliable performance under moderate conditions. It can measure temperature from 0 to 50 °C with ± 2 °C accuracy, and humidity from 20% to 90% RH with $\pm 5\%$ RH accuracy. The sensor communicates over a single-wire protocol and is compatible with Arduino libraries, making integration straightforward.

Soil moisture sensor: The capacitive soil moisture sensor is a key component for assessing soil hydration levels in the smart greenhouse system. Unlike resistive sensors, the capacitive variant measures the dielectric permittivity of the soil to determine moisture content, making

it less prone to corrosion and wear. The sensor outputs an analog voltage that correlates with the moisture level: lower values indicate dry soil, and higher values indicate wet conditions. It is interfaced with an Arduino using the analog input pin and requires calibration to map voltage levels to real moisture percentages. In our system, if the sensor value drops below 100, it triggers an alert and activates the servo motor (water pump) to irrigate the plants.

Raspberry Pi 3: The Raspberry Pi 3 serves as the central MQTT broker and local publisher in our system. It runs a Mosquitto MQTT broker instance and processes incoming sensor data from the publishing Arduino. The Raspberry Pi 3 also republishes this data to external cloud services like Adafruit IO and allows terminal-based command input for remote actuation. Python scripts manage the message handling and data routing.

Arduino Uno WiFi Rev2: Two Arduino Uno WiFi Rev2 boards are used in this system—one as a sensor node (publisher) and the other as an actuator node (subscriber). These microcontrollers support WiFi connectivity, making them ideal for MQTT communication. The publisher Arduino collects data from the DHT11 and soil moisture sensor and sends it to the broker. The subscriber Arduino responds to soil moisture alerts by activating the buzzer or servo motor and can receive remote ON/OFF commands via MQTT from cloud interfaces.

Technological approaches used

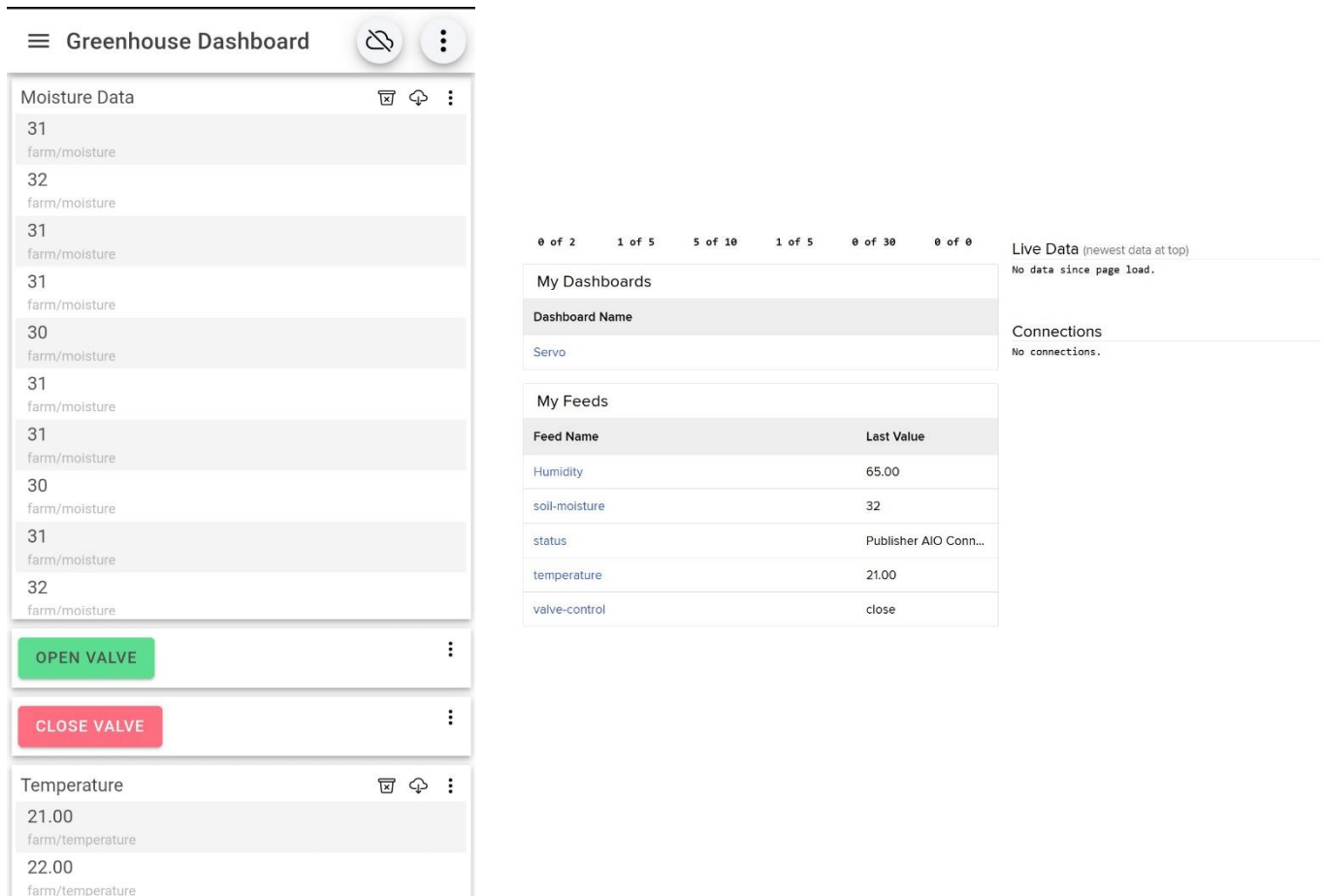
- MQTT protocol for communication
- Raspberry Pi 3 (Python-based MQTT Broker and Publisher)
- Arduino Uno WiFi Rev2 (C++/Arduino IDE)
- DHT11 for temperature and humidity
- Capacitive soil moisture sensor
- Servo motor as water pump emulator
- Buzzer for alert
- Adafruit IO for cloud visualization and control
- MQTT Dashboard App and Terminal for local/remote control

5 Results:

- The publisher Arduino accurately sensed soil moisture, temperature, and humidity values and published them in real-time.
- The Raspberry Pi 3 broker received, parsed, and republished data correctly to both local and cloud-based subscribers.
- The subscriber Arduino responded instantly to low moisture alerts by activating the buzzer, providing a clear audible warning.

Team Name

- Remote control via Adafruit IO, MQTT Dashboard App, and terminal worked as intended. When a user sent a "motor ON" command, the subscriber activated the servo motor to simulate irrigation and disabled the buzzer.
- The system proved responsive and stable during repeated testing under different environmental conditions.



6 Implementation

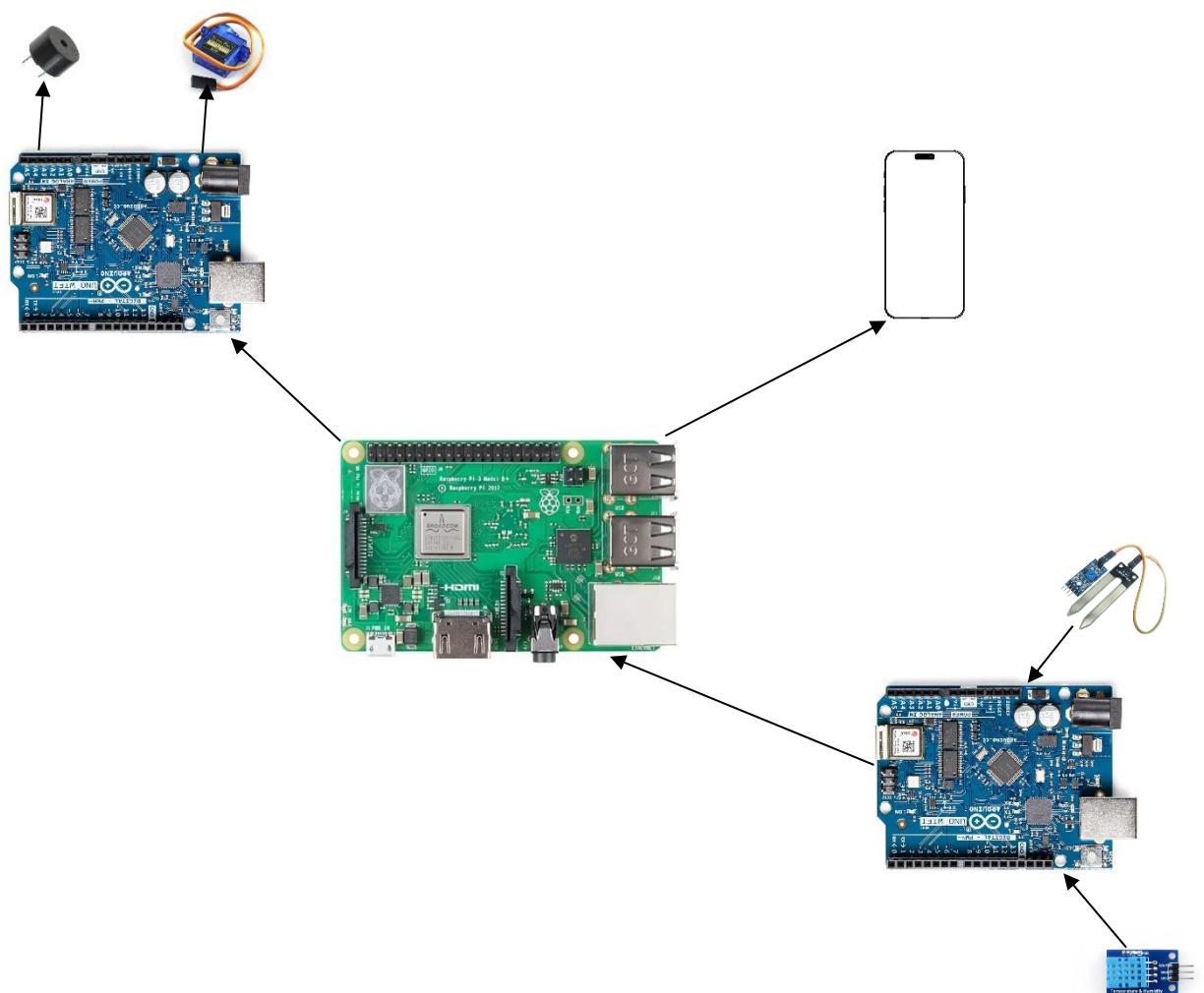
The system uses MQTT (Message Queuing Telemetry Transport), a lightweight messaging protocol ideal for IoT applications due to its low bandwidth usage and publish/subscribe architecture. In this project, the Raspberry Pi 3 runs a Mosquitto broker, acting as the central hub for message exchange. The Arduino Uno WiFi Rev2 boards use MQTT libraries compatible with C/C++ in the Arduino IDE to publish and subscribe to sensor data.

On the publisher side, the Arduino collects data from the DHT11 and soil moisture sensors and formats the readings as JSON payloads. These messages are sent via MQTT to the broker using the PubSubClient library. The subscriber Arduino listens for messages on the moisture

Team Name

topic. If the value received is below a defined threshold (e.g., 100), it triggers actions such as turning on a buzzer or activating a servo motor, simulating water irrigation.

Remote control is achieved by allowing external clients (Adafruit IO, MQTT Dashboard App, terminal) to publish control messages to a dedicated topic, which the subscriber Arduino reads and responds to accordingly.



7. Conclusion

In conclusion, the smart greenhouse project effectively combines IoT technologies with agriculture by utilizing the Arduino Rev 2 microcontroller and the MQTT protocol for efficient communication. The integration of MQTT Pane allowed seamless monitoring and control of environmental parameters such as temperature, humidity, and soil moisture in real-time. This setup enables automated responses to changing conditions, optimizing plant growth while minimizing manual intervention. The project highlights the power of lightweight messaging protocols like MQTT in creating scalable and responsive smart farming solutions, demonstrating a promising approach to modern, connected greenhouse management.